1. Difference Between Propositional and Predicate Logic

Definition :

- Propositional Logic (PL) deals with simple propositions that are either True or False. It does not consider the internal structure of objects.
- Predicate Logic (First Order Logic – FOL) extends PL by using predicates, variables, functions and quantifiers to reason about objects and their relations.

Comparison Table :

Feature	Propositional Logic (PL)	Predicate Logic (FOL)
Basic Unit	Proposition (e.g. P, Q)	Predicate with arguments (e.g. P(x), Loves(x, y))
Structure	Has no internal structure	Has internal structure (objects, relations)
Variables	No variables	Uses variables (x, y, z, ...)
Quantifiers	No quantifiers	Uses ∀ (for all), ∃ (there exists)
Expressiveness	Less expressive	More expressive
Knowledge Representation	Facts about propositions	Facts about objects and their relations
Example	P: It is raining. Q: The ground is wet.	Rains(x): x is raining. Wet(y): y is wet. ∀x (Rains(x) → Wet(x))
Reasoning	Simpler, decidable	More complex, semi-decidable
Applications	Digital circuits, simple AI problems	Natural language understanding, knowledge bases, expert systems

Examples :

(i) Propositional Logic

- P : It is raining.
- Q : The ground is wet.
- Rule : P → Q
- Fact : P

Conclusion : Q (by Modus Ponens)

(ii) Predicate Logic (FOL)

- Rains(x) : x is raining.
- Wet(x) : x is wet.
- Rule : ∀x (Rains(x) → Wet(x))
- Fact : Rains(Chennai)

Conclusion : Wet(Chennai) (Universal Instantiation + Modus Ponens)

Key Takeaway :
PL reasons about facts.
FOL reasons about objects, their properties and relations.

2. Inference in First Order Logic (FOL)

Definition :
Inference in FOL is the process of deriving new sentences (conclusions) from a knowledge base (set of FOL sentences) using rules of inference that are sound and valid.

Common Rules of Inference in FOL :

1. Universal Instantiation (UI) : From ∀x P(x), infer P(c) for any constant c.
2. Existential Instantiation (EI) : From ∃x P(x), infer P(c) for a new constant c.
3. Universal Generalization (UG) : From P(c) (c arbitrary), infer ∀x P(x).
4. Existential Generalization (EG) : From P(c), infer ∃x P(x).
5. Modus Ponens (MP) : From P and (P → Q), infer Q.
6. And Elimination (∧E) : From P ∧ Q, infer P (or Q).
7. And Introduction (∧I) : From P and Q, infer P ∧ Q.

Example :

Knowledge Base (KB) :

1. ∀x (Human(x) → Mortal(x))
2. Human(Socrates)

Query : Mortal(Socrates)

Proof (using inference rules) :

Step	Sentence	Rule
1	∀x (Human(x) → Mortal(x))	Premise
2	Human(Socrates)	Premise
3	Human(Socrates) → Mortal(Socrates)	UI from (1)
4	Mortal(Socrates)	MP from (2) and (3)

Conclusion : Mortal(Socrates) is inferred from the KB.

3. Bayesian Reasoning

Definition :
Bayesian reasoning is a method of reasoning under uncertainty based on Bayes' Theorem. It allows us to update the probability of a hypothesis given new evidence.

Bayes' Theorem :

$$P(H|E) = \frac{P(E|H)\,P(H)}{P(E)}$$

where,

- P(H|E) : Posterior probability (probability of H given E)
- P(E|H) : Likelihood (probability of E given H)
- P(H) : Prior probability (initial belief in H)
- P(E) : Evidence probability (total probability of E)

Example : Medical Diagnosis

A disease D affects 1% of a population.
A test T is 95% accurate :

- P(T+|D) = 0.95 (test positive given disease)
- P(T+|¬D) = 0.05 (test positive given no disease)

If a person tests positive (T+), what is the probability that he actually has the disease (D)?

Solution :

P(D) = 0.01 , P(¬D) = 0.99
P(T+|D) = 0.95 , P(T+|¬D) = 0.05

Using Bayes' Theorem,

$$P(D|T+) = \frac{P(T+|D)\,P(D)}{P(T+|D)\,P(D) + P(T+|\neg D)\,P(\neg D)}$$

$$= \frac{0.95 \times 0.01}{(0.095 \times 0.01) + (0.05 \times 0.99)} = \frac{0.0095}{0.0095 + 0.0495} = 0.161$$

Therefore, P(D|T+) ≈ 0.161 or 16.1%

Q. Explain NLP issues. Also explain Expert System architecture.

Ans. 1 Natural Language Processing (NLP) – Issues

Natural Language Processing (NLP) is a field of AI that enables computers to understand, interpret and generate human language. Despite significant progress, NLP faces several challenges.

Major Issues in NLP :

1. Ambiguity : Words and sentences can have multiple meanings.
   Example : "bank" → can mean financial bank or river bank.
2. Context Understanding : The meaning of a word depends on the context in which it is used.
   Example : "I saw him on the bank" (needs context to decide meaning).
3. Language Complexity : Human language has complex grammar, idioms, sarcasm, slang etc.
   Example : "That's cool!" (may mean appreciation or indifference).
4. Data Sparsity : Many words/phrases occur rarely, lack of sufficient training data affects accuracy.
5. Variations in Language : Different languages, accents, dialects and informal expressions make processing difficult.
6. World Knowledge : Machines lack real world knowledge required to understand implicit meaning.
7. Computational Cost : NLP tasks like parsing, translation and summarization require high computational resources.

NLP TASKS (Examples)

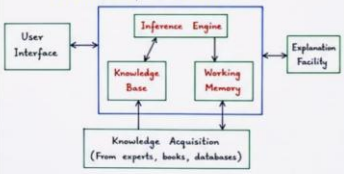
Example :

- In Machine Translation, the word "right" may mean "correct" or "right side".
- In Sentiment Analysis, the sentence "This movie was sick!" may be positive (very good) in informal usage.

Ans. 2 Expert System Architecture

An Expert System is an AI program that simulates the decision-making ability of a human expert in a specific domain. It uses knowledge and reasoning to solve complex problems.

Architecture / Components :



1. User Interface : Allows user to input facts, ask questions and get solutions.
2. Knowledge Base : Contains domain knowledge, facts, rules and heuristics.
3. Inference Engine : Applies rules on facts in working memory to derive new facts or conclusions.
4. Working Memory : Stores facts about the current problem.
5. Explanation Facility : Explains how the system reached a particular conclusion.
6. Knowledge Acquisition Module : Helps in gathering and updating knowledge from experts or other sources.

Example :

- In a Medical Expert System, symptoms entered by the user are matched with rules in the knowledge base.
- The inference engine concludes the possible disease and suggests treatment.

1. Hill Climbing

Definition : Hill Climbing is a local search algorithm that starts with an initial state and moves to a neighboring state that has a better value according to an evaluation function. It continues until no better neighbor is found.

Explanation / Working :

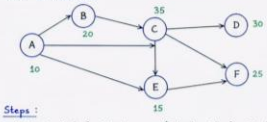
1. Start from an initial state.
2. Evaluate all the neighboring states using an evaluation function.
3. Move to the neighbor with the best (highest) value.
4. Repeat the process.
5. Stop when no neighbor has a better value.

Characteristics :

- Uses only local information.
- Greedy strategy.
- May get stuck at local maxima, plateaus or ridges.

Example :

Consider the following state space. The value in each state represents the evaluation (to be maximized). Start at state A.



Steps :

- Start at A (10). Neighbors → {B(20), C(35), E(15)} Best is C (35). Move to C.
- From C, neighbors → {B(20), D(30), E(15), F(25)} Best is D (30). Move to D.
- From D, neighbor → {C(35)} which is not better. So, stop.

Therefore, Hill Climbing terminates at D with value 30.
Note: Global maximum is 35 at C, but algorithm got stuck at local maximum (D = 30).

2. Heuristic Function

Definition : A Heuristic function is a function h(n) that estimates the cost from the current node n to the goal state. It is used to guide the search towards the goal and to reduce the search space.

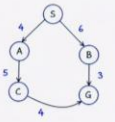
Explanation / Key Points :

1. Heuristic gives an estimate, not exact cost.
2. It helps in informed search algorithms like Greedy Best-First Search and A* Search.
3. A good heuristic is :
   - Admissible : Never overestimates the true cost to the goal. (h(n) ≤ h*(n))
   - Consistent : For every node n and its successor n', h(n) ≤ c(n,n') + h(n').
4. Better heuristic → fewer nodes expanded → efficient search.
5. If h(n) = 0 for all n, the search becomes equivalent to Uniform Cost Search.

Example :

Consider a path finding problem from Start (S) to Goal (G). The straight-line distance (in km) to the goal is used as heuristic h(n).

Node	h(n) (Estimated distance to Goal)
S	14
A	10
B	7
C	4
G	0



Using h(n) the search will prefer nodes that are closer to goal. For example, from S it will choose B (h=7) over A (h=10). Finally it reaches G where h(G) = 0.

Hence, heuristic function provides an intelligent estimate to guide the search towards the goal efficiently.

Ans. 1. Supervised and Unsupervised Learning

Machine Learning techniques are broadly divided into two types : Supervised Learning and Unsupervised Learning.

- **Supervised Learning :**
  - Definition : In supervised learning, the algorithm is trained using labelled data. Each training example has an input and a corresponding output.
  - Goal : To learn a mapping from inputs to outputs and predict the output for new unseen data.
  - How it works : The model learns a function $f(x)$ from training set $\{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$ and predicts y for a new x.
  - Types : Classification and Regression.
  - Example :
    - Email spam detection (Spam / Not spam)
    - Predicting house price from features.
- **Unsupervised Learning :**
  - Definition : In unsupervised learning, the algorithm is given unlabelled data. It finds patterns or structure in the data without any target output.
  - Goal : To discover hidden patterns, groups or relationships in the data.
  - How it works : The model explores the data and clusters or reduces dimensions based on similarity.
  - Types : Clustering and Dimensionality Reduction.
  - Example :
    - Customer segmentation using K-means clustering.
    - Grouping similar documents.

Difference Between Supervised and Unsupervised Learning

Feature	Supervised Learning	Unsupervised Learning
Data	Labelled data	Unlabelled data
Goal	Predict output	Discover patterns
Output	Specific (Class/Value)	General (Clusters/Groups)
Examples	Classification, Regression	Clustering, Association
Algorithms	Linear Regression, SVM, Decision Trees	K-means, Hierarchical Clustering, PCA

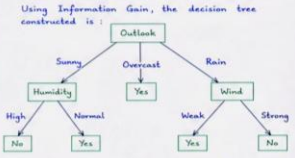
Ans. 2. Decision Trees

A Decision Tree is a flowchart-like structure in which internal nodes represent tests on attributes, branches represent outcomes of the tests and leaf nodes represent class labels or decisions.

- **Construction of Decision Tree :**
  1. Start with the entire training set at the root.
  2. Select the best attribute using a measure such as Information Gain or Gini Index.
  3. Split the data into subsets based on the attribute values.
  4. Repeat the process recursively for each subset.
  5. Stop when all records in a node belong to the same class or no attributes are left.
- **Advantages :**
  - Easy to understand and interpret.
  - Requires little data preparation.
  - Handles both numerical and categorical data.
  - Can model non-linear relationships.
- **Limitations :**
  - Can be unstable (small change in data may change the tree).
  - Prone to overfitting.
- **Example :** Consider the following training data to decide whether a person will play Tennis.

Outlook	Temperature	Humidity	Wind	Play Tennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes

Using Information Gain, the decision tree constructed is :



This tree can be used to predict "Play Tennis" for any new instance.

Q. Explain Neural Networks. Explain SVM.

1. Neural Networks

Definition : A Neural Network (NN) is a computing model inspired by the human brain. It consists of interconnected neurons (nodes) organized in layers. NN learns from examples and is used for pattern recognition, classification and prediction.

Architecture :

A typical feed-forward neural network has three types of layers.

1. Input Layer – accepts input features.
2. Hidden Layer(s) – performs computation using weights and activation function.
3. Output Layer – produces final output.

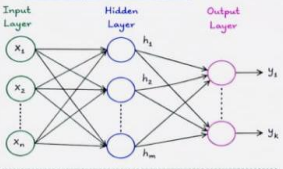
Working (Forward Propagation) :

1. Each input is multiplied by a weight.
2. The weighted inputs are summed and bias is added.
3. Activation function is applied to produce output.
4. The process continues layer by layer till output layer.

Common Activation Functions :

Sigmoid $f(z) = \frac{1}{1+e^{-z}}$, Tanh $f(z) = \tanh(z)$, ReLU $f(z) = \max(0, z)$

Structure of a Neural Network



Net input to hidden neuron j : $z_j = \sum_{i=1}^{n} x_i w_{ij} + b_j$

Output of hidden neuron : $h_j = f(z_j)$ (activation function)

Example :

Consider XOR problem.

Input 1	Input 2	Desired Output
0	0	0
0	1	1
1	0	1
1	1	0

A neural network with one hidden layer can learn this non-linear relationship which is not possible using a single linear model.

2. Support Vector Machine (SVM)

Definition : Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. It works by finding the optimal hyperplane that best separates the classes with the maximum margin.

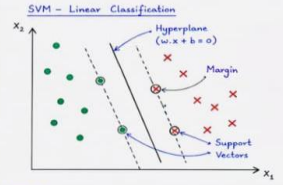
Key Points :

1. Finds the best decision boundary (hyperplane).
2. Maximizes the margin between classes.
3. Only a subset of training points called Support Vectors lie on the margin.
4. Works with linear and non-linear data (using kernels).
5. Used in classification, regression, and anomaly detection.

Types of SVM :

1. Linear SVM – for linearly separable data.
2. Kernel SVM – for non-linearly separable data using kernel trick.

SVM – Linear Classification



Example :

Given two classes in 2D space. SVM finds the line that separates them with maximum margin. The points lying on the margin are support vectors.

In Kernel SVM, data is first mapped to a higher dimensional space using a kernel function (e.g., RBF, Polynomial) and then a hyperplane is found.

1. Explain First Order Logic (FOL)

Definition :

First Order Logic (FOL) is a formal logic that allows reasoning about objects, their properties and relations. It extends propositional logic by using predicates, functions, variables and quantifiers.

Key Components :

- Constants : Specific objects (e.g., John, Socrates)
- Variables : Range over objects (e.g., x, y)
- Predicates : Properties or relations (e.g., Human(x), Loves(x, y))
- Functions : Map objects to objects (e.g., Father(x))
- Quantifiers : ∀ (for all), ∃ (there exists)
- Logical Connectives : ¬, ∧, ∨, →, ↔

Syntax (well-formed formula) :

FOL formula ::= Predicate | ¬FOL | FOL ∧ FOL | FOL ∨ FOL | FOL → FOL | FOL ↔ FOL | ∀x FOL | ∃x FOL

Examples :

1. ∀x (Human(x) → Mortal(x))
   (For all x, if x is human then x is mortal.)
2. Human(Socrates)
   (Socrates is a human.)
3. Mortal(Socrates)
   (Socrates is mortal.)
4. ∃x (Student(x) ∧ Loves(x, AI))
   (There exists a student who loves AI.)

Knowledge Base Example :

1. ∀x (Human(x) → Mortal(x))
2. Human(Socrates)

Query : Mortal(Socrates)

Explanation :

From the KB, anyone who is human is mortal.
Socrates is human. Therefore, Socrates is mortal.

2. Explain Resolution / Theorem Proving

Definition :

Resolution is a rule of inference used in theorem proving. It works by deriving new clauses from existing clauses until either an empty clause (□) is derived (proof succeeds) or no new information can be derived (proof fails).

Key Points :

- KB is converted to Conjunctive Normal Form (CNF).
- Negate the query and add it to KB.
- Repeatedly apply resolution rule to derive new clauses.
- If empty clause (□) is obtained ⇒ query is entailed (proved).
- Otherwise, query is not entailed.

Resolution Rule :

$(A \vee P), (\neg P \vee B) \;\therefore (A \vee B)$ where P is any literal.

Example :

KB : 1. ∀x (Human(x) → Mortal(x))
2. Human(Socrates)

Query : Mortal(Socrates)

Step 1 : Convert KB and ¬Query to CNF

- ∀x (Human(x) → Mortal(x)) ≡ ∀x (¬Human(x) ∨ Mortal(x))
  After removing ∀ : (¬Human(x) ∨ Mortal(x))
- Human(Socrates)
- ¬Mortal(Socrates) (negation of query)

Step 2 : List of clauses

C1 : (¬Human(x) ∨ Mortal(x))
C2 : (Human(Socrates))
C3 : (¬Mortal(Socrates))

Step 3 : Apply resolution

Resolve	From Clauses	Result	Explanation
C1 and C2	(¬Human(x) ∨ Mortal(x)) and (Human(Socrates))	(Mortal(Socrates))	Unify x with Socrates, resolve on Human
(Mortal(Socrates)) and C3	(Mortal(Socrates)) and (¬Mortal(Socrates))	□ (empty clause)	Resolve on Mortal(Socrates)

Since empty clause is derived, the query Mortal(Socrates) is proved.
Hence, the KB entails the query.

3. Explain Bayesian Network

Definition :

A Bayesian Network (BN) is a graphical model that represents a set of random variables and their conditional dependencies using a Directed Acyclic Graph (DAG). It allows reasoning under uncertainty using probability.

Key Points :

- Nodes represent random variables.
- Directed edges represent direct (causal) dependencies.
- Each node has a Conditional Probability Table (CPT).
- The joint probability is the product of all CPTs.

Joint Probability Formula :

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

where $\text{Parents}(X_i)$ are the direct predecessors of X_i.

Inference Example :

Find P(Burglary = T | JohnCalls = T, MaryCalls = T)
This can be computed using Bayes' theorem and the CPTs via enumeration or variable elimination.

Example : Burglary Alarm Network

P(B)

B	P(B)
T	0.001
F	0.999

P(E)

E	P(E)
T	0.002
F	0.998

P(A | B, E)

B	E	P(A=T)	P(A=F)
T	T	0.95	0.05
T	F	0.94	0.06
F	T	0.29	0.71
F	F	0.001	0.999

P(J | A)

A	P(J=T)	P(J=F)
T	0.90	0.10
F	0.05	0.95

P(M | A)

A	P(M=T)	P(M=F)
T	0.70	0.30
F	0.01	0.99

BNs are widely used in diagnosis, prediction, decision making, and other real-world AI applications.

1. Explain Minimax Algorithm with an Example.

Definition :

Minimax is a game playing algorithm used in two player, zero-sum games. One player tries to maximize the score (MAX) while the other tries to minimize the score (MIN). The algorithm explores the game tree and backtracks the values to choose the optimal move.

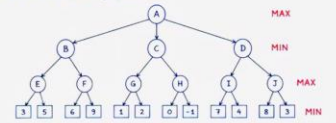
Algorithm (Minimax) :

1. Generate the game tree from the current state.
2. Assign utility values to all terminal (leaf) nodes.
3. Propagate the values upward :
   - MAX level → take maximum of children.
   - MIN level → take minimum of children.
4. The value at the root node gives the best achievable value for MAX.
5. Choose the move leading to that value.

Therefore, the minimax value of the root (A) is 7.
So, MAX should choose move leading to node D.

Example :

Consider the following game tree.



Step 1 : Evaluate leaf nodes.

Step 2 : MAX level (E, F, G, H, I, J)

E = max(3, 5) = 5, F = max(6, 9) = 9, G = max(1, 2) = 2,
H = max(0, -1) = 0, I = max(7, 4) = 7, J = max(8, 3) = 8

Step 3 : MIN level (B, C, D)

B = min(5, 9) = 5, C = min(2, 0) = 0, D = min(7, 8) = 7

Step 4 : Root (A) MAX level

A = max(5, 0, 7) = 7

2. Explain Alpha-Beta Pruning with an Example.

Definition :

Alpha-Beta pruning is an optimization of minimax algorithm. It eliminates the branches that cannot affect the final decision. It reduces the number of nodes evaluated, thus saving time and space.

Key Ideas :

- Alpha (α) → best (highest) value found so far for MAX.
- Beta (β) → best (lowest) value found so far for MIN.
- If α ≥ β, remaining children need not be explored (pruned).

Algorithm (Alpha-Beta Pruning) :

1. Start the search with α = −∞, β = +∞.
2. Traverse the game tree depth-first (left to right).
3. At MAX node : update α = max(α, value).
   At MIN node : update β = min(β, value).
4. If α ≥ β at any node, prune the remaining children.
5. Continue until the value of the root is determined.

Result :

- Pruned nodes : H and its children (0, -1) and J's right child (3).
- Minimax value at root A = 7
- Best move for MAX → choose D.

Example : Consider the same tree.

Step-by-step evaluation (left to right) :

Node	Type	Value	α	β	Action / Note
A	MAX	–	−∞	+∞	Start
B	MIN	–	−∞	+∞	
E	MAX	max(3,5) = 5	−∞	5	return 5 to B
F	MAX	max(6,9) = 9	−∞	5 → 5	return 9 to B
B	MIN	min(5,9) = 5	−∞	+∞ → 5	return 5 to A
A	MAX	max(−∞,5) = 5	5	+∞	α at A = 5
C	MIN	–	5	+∞	
G	MAX	max(1,2) = 2	5	+∞	β at C = 2
D	MIN	–	5	+∞	
I	MAX	max(7,4) = 7	5	7	β at D = 7

Since β (2) ≤ α (5) → prune H and its children.

Since β (7) ≤ α (5) is false, but next child J:

After evaluating J, α (5) ≥ β (7) i.e., 7) = 7, since α (5) ≥ β (7) is false.
Actually pruning occurs because after I, α (5) ≥ β (7) is false.
However at J, during exploration first leaf 8 gives β = 8 (but β already 7 from I).
So when visiting J, α (5) ≥ β (7) is true → prune remaining child (3).

1) BFS (Breadth First Search)

Definition : BFS explores the graph level by level. It expands all the nodes at the present depth before moving to the next depth.

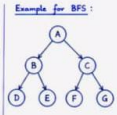
Algorithm (BFS) :

1. Start from the initial node.
2. Put the start node into a queue.
3. Remove the front node from queue and visit it.
4. Insert all its unvisited adjacent nodes into the queue.
5. Repeat steps 3-4 until the goal is found or queue becomes empty.

Properties :

- Complete.
- Optimal (for unit cost).
- Space complexity is high.

Example for BFS :



Starting node = A, Goal = G

BFS traversal order :

A B C D E F G

(Level order)

2) DFS (Depth First Search)

Definition : DFS explores as far as possible along each branch before backtracking.

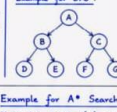
Algorithm (DFS) :

1. Start from the initial node.
2. Visit the node and mark it.
3. Recur for an unvisited adjacent node.
4. If no adjacent node is left, backtrack.
5. Repeat until goal is found or all nodes are visited.

Properties :

- Not complete in infinite depth.
- Not optimal.
- Space complexity is less.

Example for DFS :



Starting node = A, Goal = G
One possible DFS order :

A B D E C F G

(Depth wise)

3) A* Search

Definition : A* (A-star) search is an informed search algorithm that uses both the actual cost from start to a node and heuristic estimate from the node to the goal.

Evaluation Function : $f(n) = g(n) + h(n)$

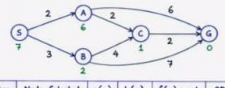
where, $g(n)$ = cost from start to n
$h(n)$ = heuristic estimate from n to goal
$f(n)$ = estimated total cost through n

Algorithm (A* Search) :

1. Put the start node in OPEN list with $f(n) = g(n) + h(n)$.
2. Choose the node with smallest $f(n)$ from OPEN.
3. Move it to CLOSED list.
4. Expand it and generate successors.
5. For each successor, compute $f(n)$.
   Add to OPEN list if not present or if a better path is found.
6. Repeat until goal node is selected.
7. Reconstruct path using parent pointers.

Example for A* Search :

Heuristic values (h) are shown in green.
Edge costs are shown in black.



Step	Node Selected	g(n)	h(n)	f(n) = g+h	OPEN List (f)
0	S	0	7	7	A(8), B(5)
1	B	3	2	5	A(8), C(8), G(10)
2	A	2	6	8	C(4), G(8)
3	C	6	1	7	G(8)
4	G	8	0	8	Goal reached

Optimal path found : S → B → C → G
Total cost = 8

4) Greedy Search (Best-First Search)

Definition : Greedy search is an informed search algorithm that always expands the node that is closest to the goal according to a heuristic function. It uses only heuristic value.

Evaluation Function : $f(n) = h(n)$

Algorithm (Greedy Search) :

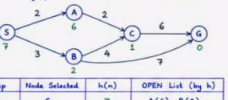
1. Put the start node in OPEN list.
2. Select the node with smallest $h(n)$.
3. Move it to CLOSED list.
4. Expand it and add its successors to OPEN.
5. Repeat until goal is found.

Properties :

- Fast (in many cases).
- Not optimal (may not give least cost path).
- Not complete in infinite search space.

Example for Greedy Search :

(Same graph and heuristic values as above)



Step	Node Selected	h(n)	OPEN List (by h)
0	S	7	A(6), B(2)
1	B	2	C(1), A(6), G(0)
2	G	0	Goal reached

Path found by Greedy : S → B → G
Total cost = 10 (Not optimal, since optimal cost is 8)